Universität Münster

Autonomous Intelligent Systems Group
www.uni-muenster.de/AISystems
Universität Münster
Einsteinstraße 62, 48149 Münster

Autonomous Systems and Mobile Robots

Exercise Sheet 1 – Summer Term 2026

Professor
Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor
Janosch Bajorath
j.bajorath@uni-muenster.de

First Contact with ROS 2 — Quadruped Demo and Turtle Drawing

This sheet has two parts. **Part I** (Tasks 1–3) is the tutorial session: you explore a running ROS 2 stack by inspecting the Unitree Go1 quadruped in Gazebo simulation. **Part II** (Tasks 4–7) is take-home: you build your own ROS 2 package from scratch and make a turtle draw a five-pointed star using three communication patterns — topics, services, and actions — then wire the whole system together with a launch file.

Work through Part I in order during the session. Part II is designed to be completed independently; all the guidance you need is on the sheet, supplemented by links to the official ROS 2 documentation. Students who cannot attend the session are expected to work through both parts independently.

Part I – Quadruped Exploration

We provide you with a ROS 2 package that brings up the Unitree Go1 quadruped in a Gazebo simulation, and a small helper package (`go1_control`) that commands the robot between two poses: sitting and standing. Your job is *not* to write nodes — it is to *explore* what is already running, discover how the pieces talk to each other, and watch a PD controller in action with `rqt`.

Work through the tasks in order. Task 2 builds on the node graph you mapped in Task 1; Task 3 builds on the plots you set up in Task 2. Along the way you are expected to *figure things out* rather than follow a recipe — the hints point you at the right ROS 2 command-line tools; the exact invocation and interpretation are up to you.

Task 1 – Exploring the ROS 2 Graph

Before you command the robot to do anything, find out what is already running when the simulation is launched. ROS 2 ships a set of small command-line tools (`ros2 node`, `ros2 topic`, `ros2 interface`) and a graphical companion (`rqt`) that together let you inspect any running system without reading a single line of its source code. Use them.

Setup. Unzip the provided archive and place the three packages (`go1_description`, `go1_control`, `go1_interfaces`) into the `src/` directory of your ROS 2 workspace. Install dependencies, build, and launch:

```
cd ~/ros2/asmr_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install \
    --packages-select go1_description go1_interfaces
source install/setup.bash
ros2 launch go1_description robot_sim.launch.py
```

Leave that terminal running. Open a second terminal (source the workspace again) for the exploration below.

(a) **List what is running.**
Use the ROS 2 command-line tools to answer the following. Keep your answers short — one or two sentences per bullet is enough.

i. Which ROS 2 nodes are currently alive? Group them by the role they appear to play (simulation, state publishing, control, . . .).

ii. Which topics exist? Pick three that look interesting and state what you think each one carries, and why.

iii. Find the topic that carries the *measured* joint state of the robot. What is its message type? Which fields of the message are populated, and which are empty?

> **Hint.** `ros2 node list`, `ros2 topic list -t`, `ros2 topic info <topic>`, `ros2 topic echo <topic>` and `ros2 interface show <msg-type>` will get you through this. `-t` on `topic list` prints the message type next to each topic.

(b) **Visualise the node graph.**
Open `rqt_graph` and let it redraw until the view stabilises.

i. Identify the controller node. Which topic does it *subscribe* to in order to receive reference commands? Which topic does it *publish*?

ii. Which node is responsible for broadcasting the TF frames? How does it obtain the joint positions it needs?

iii. Sketch (by hand or by annotating a screenshot) the data flow from *reference command* to *robot joint state in RViz*. Mark each arrow with the topic name.

Task 2 – Commanding Poses and Watching the Controller

With the node graph from Task 1 in mind, you will now drive the robot between two poses using a pre-built helper node, and use `rqt_plot` to look at what the PD controller actually does to the joints over time.

Setup. With the Gazebo simulation from Task 1 still running, build the control package and start its nodes. Open sourced terminals as needed:

```
# Terminal B -- build and source (once)
cd ~/ros2/asmr_ws
colcon build --symlink-install --packages-select go1_control
source install/setup.bash

# Terminal C -- action server
ros2 run go1_control pose_commander

# Terminal D -- keyboard client
ros2 run go1_control keyboard_commander

# Terminal E -- joint relay (needed for plotting in part b)
ros2 run go1_control joint_relay
```

Consult the package `README` for the key bindings.

(a) **Send the robot into a pose.**
In the `keyboard_commander` terminal, press a key to command a pose (e.g. `1` for standing, `2` for crouching).

i. Observe the robot in Gazebo. Does it reach the commanded pose? Does it reach it *cleanly*, or does it wobble, overshoot, or fail to arrive at all?

ii. Run `ros2 action list -t` in a sourced terminal while `pose_commander` is running. Which action server do you see, and what is its type?

iii. Run `ros2 topic echo /joint_pid_controller/reference` and trigger a transition. Confirm that position references are being streamed. On which topic does the controller receive them, and does this match the reference input you identified in Task 1 (b)?

> **Hint.** The action server and topic both become visible as soon as `pose_commander` starts — you do not need to trigger a transition first. `ros2 action list -t` shows type information next to each action name, analogous to `ros2 topic list -t`.

(b) **Plot the positional error.**
Open `rqt` and add the **Plot** plugin (*Plugins* → *Visualization* → *Plot*).

Hint. `rqt_plot` timestamps each curve using the message header. `/joint_states` carries a `header.stamp` set to Gazebo *simulation time*, but the `control_msgs/MultiDOFCommand` on `/joint_pid_controller/reference` has no header at all — `rqt_plot` falls back to *wall-clock time* for it. Because simulation time and wall-clock time run at different rates, the two curves end up on mismatched time axes and cannot be overlaid. `joint_relay` (Terminal E) fixes this: it subscribes to both topics, extracts a single joint by name, and re-publishes both values inside the `/joint_states` callback — so both outputs share the same simulation timestamp.

The `joint_relay` node you started in Terminal E publishes the commanded and measured position of a single joint on two topics with a shared timestamp, making them directly overlayable.

i. Add the following two topics to the plot: `/joint_relay/position/data` (measured) and `/joint_relay/reference/data` (commanded). By default, the relay tracks `FL_calf_joint`.

ii. Command a crouching → standing transition (press `2`, then `1`). Capture (screenshot) the resulting plot. The *gap* between the two curves over time is the positional error of the controller. Describe its shape in words: delay, overshoot, steady-state offset, oscillation.

iii. Repeat the crouching → standing transition two or three times. Is the error behaviour *reproducible*, or does something drift between runs?

Hint. `joint_relay` looks up the joint by *name* in the `name` list of each message, so the order of joints within `/joint_states` and `/joint_pid_controller/reference` does not need to match — the `name`/`dof_names` array defines the mapping. To track a different joint, pass the `joint_name` parameter:
`ros2 run go1_control joint_relay --ros-args -p joint_name:=FR_thigh_joint`

Task 3 – Tuning the PD Gains

The controller you observed in Task 2 runs a PD law per joint: the commanded torque is $\tau = K_p\,(q_{\text{ref}} - q) + K_d\,(\dot{q}_{\text{ref}} - \dot{q})$. The gains K_p and K_d are stored per pose in the `go1_control` package. In this task you will change them and watch the effect live on the `rqt_plot` from Task 2.

Configuration file. The gains live in `go1_control/config/poses.yaml`, under the `gains:` key of each named pose. Each joint entry has the form `{p: <value>, d: <value>}`. `pose_commander` pushes these values to the running controller before every transition; editing `go1_description/config/controller_pid.yaml` has no effect while `pose_commander` is active. Default values are non-zero (hip joints: `p: 100`, `d: 2`; thigh/calf: `p: 300`, `d: 8`).

Reloading gains. After editing `poses.yaml`, restart `pose_commander` and `keyboard_commander`; the Gazebo simulation can stay running. Changes take effect on the next pose transition.

(a) **Baseline behaviour with zero gains.**
In `poses.yaml`, set `p` and `d` to `0.0` for every joint in the pose you will test (e.g. `standing`). Restart `pose_commander` and `keyboard_commander`, then trigger a transition. Watch the plot from Task 2 (b).

i. What does the commanded trajectory look like? What does the measured trajectory look like? Explain the discrepancy in one sentence.

ii. Why does the robot not fall through the floor despite receiving no actuation torque? (*Hint: what does Gazebo still apply, and what does the robot model contribute?*)

(b) **Sweep the proportional gain.**
Set $K_d = 0$ and try $K_p \in \{5,\ 50,\ 500\}$ for all joints. For each value, command a crouching $\rightarrow$ standing transition and capture the plot of commanded vs. measured position for the same joint as in Task 2.

i. Describe the qualitative effect of increasing K_p on: *rise time*, *overshoot*, *steady-state error*, *oscillation*.

ii. Which of the three values would you call "reasonable"? Justify with one sentence referencing your plots.

(c) **Add damping.**
Pick the K_p you called reasonable above. Now vary $K_d \in \{0,\ 2,\ 10\}$.

i. What does the derivative term do to the overshoot you saw with $K_d = 0$?

ii. What happens when K_d becomes too large? Describe the failure mode in one sentence.

Part II – Building Your First ROS 2 Package

In this part you write a ROS 2 package called `turtle_star` from scratch. The package contains a single Python node that drives `turtlesim` to draw a five-pointed star. You build the node incrementally: each task introduces one communication pattern on top of what you have already written. By the end, a single launch command opens the simulator and draws the star.

What is turtlesim? `turtlesim` is a lightweight 2D simulator that ships with ROS 2. It displays a small turtle on a canvas and exposes a set of topics, services, and actions that you can call to move the turtle, change its pen colour, or rotate it to a precise heading. Because `turtlesim` uses exactly the same ROS 2 interfaces as real robots, it is the standard sandbox for learning the communication patterns before applying them on hardware. Official documentation and source: `https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html`

ROS 2 Coding Standards

All code you write for this part must follow PEP 8 and the official ROS 2 Python style guide. The key conventions are:

- **Naming:** `snake_case` for files, functions, and variables; `CamelCase` for class names (e.g. `StarNode`).
- **Package name:** `snake_case` (`turtle_star`).
- **Logging:** `self.get_logger().info()`, `.warning()`, `.error()` — never `print()`.
- **Entry point:** a `main()` function with `rclpy.init()`, `rclpy.spin()`, and `rclpy.shutdown()` inside a `try/finally`, guarded by `if __name__ == '__main__':`.
- **Type hints:** encouraged on all function signatures.
- **Line length:** 99 characters (ROS 2 standard).

Reference: `docs.ros.org` → Contributing → Code Style

Task 4 – Publishing to a Topic

Local/VM setup. If you are working on a local machine or VM — not on JupyterHub — install `turtlesim` before continuing:

```
sudo apt install ros-jazzy-turtlesim
```

JupyterHub users: skip this step; the package is pre-installed in the ROS 2 image.

What is a topic? A topic is a continuous, unidirectional data stream. A *publisher* sends messages on a named channel; any number of *subscribers* can listen. There is no acknowledgement — the publisher does not know who, if anyone, is listening. This makes topics the right pattern for ongoing motion commands: the turtle moves as long as velocity messages arrive.

Publishers and subscribers are *decoupled* by the middleware. They do not call each other directly; ROS 2 routes messages between them. This decoupling is one of the key architectural benefits of the framework.

What you will build. Create a ROS 2 Python package called `turtle_star` and inside it a node class `StarNode`. In this task the node does one thing: it publishes velocity commands that move the turtle forward in a straight line for a fixed distance, then stops.

Think of the node as having two phases:

(1) **Setup** (constructor `__init__`): create the publisher and a timer that fires your callback at a fixed rate.

(2) **Runtime** (timer callback): publish a `Twist` message each time the timer fires; cancel the timer once the edge is complete.

A time-based approach — counting how many timer callbacks have fired — is the simplest way to decide when to stop. Keep this logic in mind; you will extend it across the next tasks.

Node Architecture for Part II

All setup and drawing in this part live inside `__init__`. The node runs its work sequentially — service calls, then the drawing loop — before `rclpy.spin()` is ever called. `spin()` in `main()` is a keepalive only; by the time it runs, the star is already drawn.

```
def __init__(self):
    # 1. create clients and publisher
    # 2. service calls       (Task 5)
    # 3. drawing loop        (Task 6)

def main():
    rclpy.spin(node)  # keepalive only
```

This matters for Tasks 5 and 6: both `spin_until_future_complete` and the edge-drawing loop are safe to call from `__init__` precisely because `spin()` has not started yet.

Guidance.

- Move into your workspace source directory, then create the package:

```
cd ~/ros2/asmr_ws/src
ros2 pkg create --build-type ament_python \
    --dependencies rclpy geometry_msgs \
    turtle_star
cd ~/ros2/asmr_ws
```

- Your node file lives at `turtle_star/turtle_star/star_node.py` and must be registered as a console script in `setup.py` so that `colcon` installs it.
- The topic is `/turtle1/cmd_vel` with message type `geometry_msgs/msg/Twist`. Two fields matter: `linear.x` (forward speed) and `angular.z` (rotation, positive = counter-clockwise).
- For this task, use `linear.x = 1.0` and a timer at 10 Hz. Counting 30 callbacks gives 3 s = 3.0 turtlesim units of forward motion — exactly the edge length you will need in Task 6. Cancel the timer after 30 callbacks using `self.timer.cancel()`.
- Build and test: `colcon build --symlink-install` then `source install/setup.bash`. Run `turtlesim_node` in a separate terminal first.

Hint. The official tutorial *Writing a simple publisher and subscriber (Python)* walks through the publisher pattern in full detail, including the timer callback structure and console script registration:
`docs.ros.org` → `Beginner Client Libraries` → `Pub/Sub (Python)`

Checkpoint. With `turtlesim_node` running in a separate terminal, launch your node and verify that the turtle moves forward in a straight line and then stops. The turtle starts wherever `turtlesim` placed it at launch — precise positioning comes in the next task.

Task 5 – Positioning the Turtle via a Service

What is a service? A service is a discrete, two-way interaction: one node sends a *request*, the server processes it, and the caller waits for the *response* before continuing. Unlike a topic, which is fire-and-forget, a service call is synchronous — your code does not proceed until you have received the reply.

This is the right pattern for one-time configuration events. Asking the simulator to move the turtle to a specific position is not a continuous stream; it is a single instruction that needs confirmation before you can safely continue. If you started drawing immediately after sending a teleport request, the turtle might not have moved yet.

What you will build. Extend `StarNode` with three service calls that run at node startup, before any drawing begins:

(1) **Disable the pen.** Call `/turtle1/set_pen` with the pen switched off so the repositioning move leaves no trail.

(2) **Teleport.** Call `/turtle1/teleport_absolute` to place the turtle at $x = 4.0$, $y = 5.0$, $\theta = 0$.

(3) **Re-enable the pen.** Call `/turtle1/set_pen` again with your chosen colour and width.

After these three calls complete, the drawing logic from Task 4 runs as before. Every launch now starts from a reproducible position.

Guidance.

- Create service clients in the constructor: `self.create_client(ServiceType, 'service_name')`.
- Before calling a service, wait until the server is available. `wait_for_service` returns `False` on timeout — check it and abort early rather than silently hanging:

```
if not client.wait_for_service(timeout_sec=5.0):
    self.get_logger().error('turtlesim service not available')
    return
```

- The standard `rclpy` pattern for a synchronous-style call is: send the request with `call_async()`, then hand the resulting future to `rclpy.spin_until_future_complete(self, future)`.
- **Timer ordering:** create the publisher timer as the *last* statement in `__init__`, after all three service calls have returned. If the timer is created earlier, it fires the moment `rclpy.spin()` starts — before the turtle has been repositioned.
- To discover the exact service names and message types, run `ros2 service list -t` while `turtlesim_node` is running. Inspect the request and response fields with `ros2 interface show <type>`.

Hint. The official tutorial *Writing a simple service and client (Python)* shows the full client pattern, including the `call_async` + `spin_until_future_complete` idiom: `docs.ros.org` → `Beginner Client Libraries` → `Service/Client (Python)`

Checkpoint. On launch the turtle should silently teleport to (4.0, 5.0) without leaving a trail, and then draw the same straight line from Task 4 starting from that position. Every run produces an identical result.

Task 6 – Drawing the Star via an Action

What is an action? An action is for long-running tasks where the caller needs to know when the task is *truly finished* before it can proceed. An action has three parts: a **goal** (what you want to achieve), optional **feedback** (progress updates during execution), and a **result** (the final outcome, sent once when the goal is done).

Why not rotate the turtle with a topic? You could publish an angular velocity on `cmd_vel` and wait a fixed amount of time — but then you have no reliable signal that the rotation has reached the target heading. The action's result fires *exactly* when the turtle has finished rotating. This matters here: you must not start drawing the next edge before the previous rotation is complete, or the star will be distorted.

In this task you only use the result. The feedback channel exists and you are welcome to explore it, but it is not required.

Star geometry. A five-pointed star (pentagram) is drawn by repeating the sequence *rotate to heading, move forward* five times. The turn between consecutive edges is 144° — that is $\frac{2}{5} \times 360°$, because the path visits every second vertex of the underlying regular pentagon.

The five absolute headings, measured counter-clockwise from the positive x-axis, are listed below. Use these values directly; you do not need to derive them.

Step	Heading (degrees)	Heading (radians)
1	0°	0.0000
2	144°	2.5133
3	288°	5.0265
4	72°	1.2566
5	216°	3.7699

Target edge length: **3.0 turtlesim units**. Starting from (4.0, 5.0) (set in Task 5), this draws a symmetric star centred on the canvas.

What you will build. Extend `StarNode` to replace the single straight-line draw from Task 4 with a loop that runs five times. In each iteration:

(1) Send a `RotateAbsolute` goal to `/turtle1/rotate_absolute` with the target heading.

(2) Wait for the result (the turtle has reached the heading).

(3) Publish `cmd_vel` for the required duration to draw one edge of length 3.0.

The service calls from Task 5 still run at startup before this loop.

Guidance.

- Create the action client in the constructor:
  `ActionClient(self, RotateAbsolute, '/turtle1/rotate_absolute')`.
- Wait for the server: `client.wait_for_server()`.
- Sending a goal returns a future for the *goal handle*, not the result. From the handle, call `get_result_async()` to get a second future for the actual result. Spin until each future is complete before proceeding.
- **Drawing an edge inside the loop.** After the action result arrives, publish `Twist` in a short loop. Use `rclpy.spin_once` to keep the event loop alive between publishes:

```
for _ in range(30):   # 30 x 0.1 s = 3 s = 3 turtlesim units
    self.cmd_pub.publish(twist)
    rclpy.spin_once(self, timeout_sec=0.1)
```

  The `time.sleep` warning in the hint below refers specifically to waiting for action *futures* — it does not apply to this edge loop.
- Import: `from turtlesim.action import RotateAbsolute`
  Add `turtlesim` to the `exec_depend` entries in `package.xml`.
- A common mistake: blocking the ROS 2 event loop while waiting for a future. If your node hangs, check that you are passing the node to `spin_until_future_complete`, not sleeping with `time.sleep`.
- **The timer from Task 4 is superseded.** All five edges are drawn directly inside this loop. The timer never fires because `rclpy.spin()` runs only after `__init__` returns — you can remove it or leave it as-is.

Hint. The official tutorial *Writing an action client (Python)* demonstrates exactly the two-stage future pattern (goal handle → result future) used here:
`docs.ros.org → Intermediate → Action Server/Client (Python)`

Checkpoint. Launch `turtlesim_node` and run `star_node`. The turtle teleports to (4.0, 5.0), then draws a complete five-pointed star. The shape should be symmetric and the fifth edge should return exactly to the starting point.

Task 7 – Wiring It Together with a Launch File

What is a launch file? A launch file is the entry point that wires a system of nodes together so users start the entire application with a single command. This is how real ROS 2 packages are distributed: you do not ask users to open multiple terminals and run multiple commands manually.

A Python launch file is *declarative*: it describes which nodes exist, what package they belong to, and optionally in what order or with what parameters they start. It is not a shell script. This pattern becomes essential in MiniLab 1, where your robot package must start multiple nodes (robot state publisher, controllers, sensors) in a coordinated way.

What you will build. Create a directory `launch/` inside your `turtle_star` package and add a file `star.launch.py` that starts two nodes:

(1) `turtlesim_node` from the `turtlesim` package.

(2) Your `star_node` from the `turtle_star` package.

Register the `launch/` directory in `setup.py` so that `colcon` installs it and ROS 2 can discover the launch file.

Guidance.

- A Python launch file defines a function `generate_launch_description()` that returns a `LaunchDescription` containing a list of `Node` actions. Each `Node` specifies at minimum `package` and `executable`.
- Add `launch_ros` as a dependency in `package.xml`:

```
<exec_depend>launch_ros</exec_depend>
```

  The `Node` action used in the launch file comes from `launch_ros.actions`, not from `launch`.
- The `executable` name is the key you defined in the `console_scripts` section of `setup.py`.
- The `data_files` entry in `setup.py` must include the launch directory. A typical entry looks like:

```
data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    ('share/' + package_name + '/launch',
     glob('launch/*.launch.py')),
],
```

  Replace the existing `data_files` list in `setup.py` with this block. Add `from glob import glob` at the top of the file if it is not already there.

- `turtlesim_node` starts quickly, but `star_node` calls services immediately at startup. Think about whether launch ordering guarantees turtlesim is ready. (*Hint:* the `wait_for_service(timeout_sec=5.0)` call from Task 5 already handles this — the node waits until turtlesim responds before continuing.)

Hint. The official tutorial *Creating a launch file* covers the full structure and the `data_files` registration pattern:
`docs.ros.org` → `Intermediate` → `Launch` → `Creating a launch file`